

Emergency Shelter Allocation Platform

A Nationally-Scalable, Multi-Hazard Disaster Shelter Allocation System

Pilot Region: Assam (Flood Response)

Detailed System Architecture & Step-by-Step Pipeline Explanation

Course: CSS7102 | Department of Computer Science and Engineering
Presidency University

Guide: Mr. Irfan Rajab Bhat

Team: Sharan (Team Lead), Harshil Gurjar, Suhas

1. Project Overview

The Emergency Shelter Allocation Platform is a disaster-response decision-support system designed to identify suitable emergency shelters and optimally allocate displaced populations to them during natural disasters. The system is architected to be deployable across any Indian state and across multiple hazard types (flood, cyclone, earthquake), with Assam flood response serving as the fully validated pilot implementation for this review.

The core design philosophy is a pluggable, modular pipeline: the same sequence of stages (find shelters, calculate distances, optimize allocation, persist state, visualize) runs regardless of which state or hazard is active. Only the data sources and suitability rules change per hazard type — the underlying engineering does not need to be rebuilt.

Design Principle

Built for national scale and multiple hazard types, demonstrated and backtested on a real Assam flood scenario. Architecture-first, data-second: the pipeline does not hardcode any region or disaster type.

1.1 High-Level Pipeline

The system operates as an eight-stage pipeline, described in full detail in Section 3 of this document:

1. Stage 0 — Disaster Event Initialization
2. Stage 1 — Shelter Discovery & Suitability Filtering
3. Stage 2 — Population / Demand Input
4. Stage 3 — Distance Calculation
5. Stage 4 — Optimization / Allocation (Linear Programming)
6. Stage 5 — Persistence (Database Write)
7. Stage 6 — Visualization
8. Stage 7 — Dashboard (Streamlit Application Layer)

2. System Architecture

The system is organized into five logical layers, each of which can be independently upgraded as the project scales from a prototype to a production-grade tool.

Layer	Prototype (Now)	Scale-Up Path (Future)
Data Ingestion	Bhuvan flood layers, IMD rainfall, Census/WorldPop, OSM Overpass, manual CSVs	Scheduled ingestion jobs from live government APIs
Processing	Geocoding, distance matrix, hazard-based suitability scoring	Same, with richer building/structural datasets
Optimization	PuLP linear program, vulnerability-weighted	Same core algorithm, larger-scale solvers if needed
Persistence	SQLite (3+ tables, transactional writes)	PostgreSQL + PostGIS for geo-indexed queries at scale
Application	Streamlit dashboard, Folium maps	Web + mobile app, offline-first sync

2.1 Why This Architecture Was Chosen

Three architectural decisions define this system and are each explained in more depth in later sections:

- Two-stage separation of concerns: shelter discovery (a geospatial suitability problem) is kept entirely separate from allocation (an optimization problem). This mirrors how real disaster response actually works — you must know where safe shelters exist before you can decide who goes where.
- Hazard-pluggable suitability scoring: rather than building three separate systems for flood, cyclone, and earthquake, one scoring interface is used with hazard-specific rule sets swapped in based on the active disaster event.
- Graceful degradation by design: every external dependency (routing API, building metadata, internet connectivity) has a documented fallback, because a disaster-response tool that fails when connectivity or data is imperfect is not fit for purpose.

3. Detailed Stage-by-Stage Explanation

Stage 0 — Disaster Event Initialization

Purpose: establish the context that every other stage operates within.

What happens:

9. An admin or relief coordinator selects or creates a disaster event by specifying: hazard type (flood, cyclone, or earthquake), state, affected district(s), and start date.
10. This is written to a new `disaster_events` table and assigned a unique `event_id`.
11. Every shelter search, population group, and allocation performed afterward is tagged with this `event_id`.

Why it matters: this is what allows the system to run multiple simultaneous, independent disaster responses — for example, a flood event in Assam and an earthquake event in Himachal Pradesh — without their data interfering with each other. It also means the hazard-specific suitability rules used in Stage 1 are selected automatically based on this one choice.

Output: an `event_id` that flows into every subsequent stage.

Stage 1 — Shelter Discovery & Suitability Filtering

Purpose: identify which buildings in the affected region are actually safe and appropriate to use as shelters — not just which buildings exist.

1a. Candidate building retrieval

An Overpass API query against OpenStreetMap pulls all schools, community halls, stadiums, and government buildings within the affected district's bounding box. This query is parameterized by region (not hardcoded to Assam), so the identical code path works for any Indian state. Building geometry (the polygon outline), the building type tag, and the `building:levels` tag (number of floors, where present) are retrieved alongside each candidate.

1b. Hazard-specific risk data retrieval

Based on the `hazard_type` recorded in Stage 0, the appropriate hazard dataset is pulled for the region:

Hazard Type	Risk Data Retrieved	Source
Flood	Flood inundation polygons, elevation (DEM)	Bhuvan/ISRO, CWC, SRTM DEM
Cyclone	Coastal buffer zones, cyclone-prone zone classification	IMD cyclone atlas, Coastal Regulation Zone maps
Earthquake	Seismic zone classification, landslide/liquefaction risk	BIS seismic zoning (Zones II–V)

1c. Hazard-specific suitability scoring

Each candidate building is scored using the rule set matching the active hazard type:

- Flood: discard or downrank buildings inside a mapped flood inundation polygon, or below a safe elevation threshold relative to nearby water bodies.
- Cyclone: discard or downrank buildings within the unsafe coastal buffer distance, or with structurally weak construction where building material tags are available.

- Earthquake: discard or downrank buildings on landslide-prone slopes or high soil-liquefaction zones per seismic mapping.

This is implemented as one pluggable scoring function that dispatches to a hazard-specific sub-function, rather than three duplicated pipelines — keeping the codebase maintainable as more hazards are added later.

1d. Area-based capacity estimation

Because most rural or semi-urban buildings in India have no officially published shelter capacity, capacity is estimated directly from the building's footprint using an internationally recognized humanitarian standard:

Formula

Capacity = (Floor Area x Number of Floors x Usable-Space Ratio) / Space-per-Person Standard

- Floor area is computed from the OSM building polygon, projected into a metric coordinate system (UTM) for accurate area calculation.
- Number of floors uses the building:levels tag when present; otherwise a sensible per-building-type default is applied (e.g., 2 floors for schools, 1 for community halls).
- Usable-space ratio (approximately 0.6–0.7) accounts for walls, corridors, and non-livable space, since raw floor area overstates usable capacity.
- Space-per-person standard uses the Sphere Standards benchmark of 3.5 m² per person, the same figure referenced in NDMA and international humanitarian shelter guidelines.

This removes the dependency on manually curated capacity lists, which was a major scalability blocker — the moment OSM has a building footprint, the system can estimate a defensible capacity for it, anywhere in the country.

1e. Universal filters and output

Regardless of hazard type, all candidate shelters are also filtered by a minimum capacity threshold and proximity to at least one mapped road (so they are physically reachable). The final filtered, scored, capacity-estimated shelter list is written to the shelters table, tagged with state, district, and hazard-suitability flags.

Dashboard output: a Folium map showing all candidate buildings in grey and suitable shelters highlighted in green for the selected disaster event.

Known Limitation

Structural/material tagging in OpenStreetMap is patchy in rural India, so cyclone and earthquake structural suitability will be less precise than flood elevation-based suitability, which relies on more reliable DEM data. This is stated openly as a documented limitation, not hidden.

Stage 2 — Population / Demand Input

Purpose: capture who needs to be housed.

Displaced population groups are entered — location, headcount, and vulnerability flags (elderly, children, disabled, pregnant) — and tagged with the active event_id. Each group is inserted into the population_groups table with status set to pending.

For the Assam pilot, this stage is populated using real historical displacement figures from an actual Assam flood event (rather than synthetic numbers), which is what enables the backtesting exercise described in Section 4.

Stage 3 — Distance Calculation

Purpose: determine how far each population group is from each candidate shelter — using real road distance wherever possible.

12. Primary method: for every pending population group, a call is made to OpenRouteService (ORS) to compute real road-network distance and travel time to each candidate suitable shelter.
13. Fallback method: if ORS is unreachable — due to lost internet connectivity, hitting the free-tier rate limit, or OSM road data being too sparse in that area to compute a route — the system falls back to straight-line (haversine) distance, and flags that value as 'estimated' rather than 'routed' in the data.
14. Only the top-N nearest suitable shelters per population group are retained, rather than a full cross-product, to keep the optimization problem in Stage 4 computationally small and fast.

This dual-path design is intentional: disaster zones routinely lose connectivity, and rural road mapping in OpenStreetMap is inconsistent, so a system that only works when both are perfect is not fit for real deployment.

Stage 4 — Optimization / Allocation (Linear Programming)

Purpose: decide who goes to which shelter, optimally.

15. The optimizer reads live `current_occupancy` from the shelters table — not static starting capacity — so repeated allocation runs during an ongoing event correctly account for people already housed.
16. A linear program is constructed using PuLP: minimize total distance traveled across all assignments, subject to (a) no shelter exceeding its capacity, and (b) every population group being fully assigned wherever total capacity allows.
17. Vulnerability-weighted priority is built into the objective function, so elderly, disabled, pregnant, or child-heavy groups are preferentially assigned to closer or less-crowded shelters.
18. The solver runs and returns one of two outcomes: every group is successfully housed, or — if total demand exceeds total available capacity — the model explicitly reports unmet demand as a waitlist, rather than silently overcrowding shelters or failing outright.

Output: an allocation table of group, shelter, number of people, and distance for each assignment.

Stage 5 — Persistence (Database Write)

Purpose: make every allocation permanent, auditable, and safe against partial failure.

For each allocation, three operations happen inside a single SQLite transaction:

19. Insert a row into the allocations table (the audit trail: which group went to which shelter, how many people, when).
20. Update `shelters.current_occupancy` by adding the newly allocated headcount.
21. Update `population_groups.status` to allocated.

Because all three operations are wrapped in one transaction, a failure partway through causes a full rollback — the system never ends up in a state where an allocation is logged but occupancy wasn't updated, or vice versa. This matters in practice: if a second wave of displacement arrives mid-event, rerunning Stages 2 through 5 correctly treats already-allocated people and partially filled shelters as the new starting state, rather than resetting to zero.

Stage 6 — Visualization

Purpose: make the current state of the response legible at a glance.

The `map_view` module reads current database state for the active disaster event and renders a Folium map showing shelters as colour-coded markers (green for low occupancy through red for near/at capacity), population group origin points, lines connecting each group to its assigned shelter, and any unallocated or waitlisted groups marked distinctly. This map is regenerated live on every dashboard reload, so it always reflects current data rather than a stale snapshot.

Stage 7 — Dashboard (Streamlit Application Layer)

Purpose: tie every stage into one usable interface for a relief coordinator.

- Top of page: select or create a disaster event — hazard type, state, and district.
- Section 1: view or upload population group data for the active event.
- Section 2: view Stage 1 output — suitable shelters plotted on the map, filterable by hazard type.
- A single 'Run Allocation' button triggers Stages 3 through 5 in sequence.
- Section 3: results — the allocation table, a shelter-occupancy bar chart, an unmet-demand alert where applicable, and the updated map with allocation lines drawn.
- Section 4: a backtest view comparing model output against the actual historical Assam flood response, for review credibility.

4. Validation Strategy: Backtesting Against a Real Event

To avoid the system being judged as a synthetic-data demo, its core allocation logic is validated against a real, publicly documented Assam flood event. Actual reported figures on affected districts and approximate displaced population counts are used as Stage 2 input, and actual shelters known to have been used are compared against the system's own Stage 1 suitability output and Stage 4 allocation.

This produces a short, defensible accuracy analysis: does the model's allocation broadly align with what relief authorities actually did, and where does it diverge, and why. A sensitivity analysis is also run to show how allocation outcomes shift if shelter capacity estimates (Section 3, Stage 1d) are off by roughly 10 to 20 percent, since real-world data is never perfectly precise.

5. Database Schema Summary

Table	Key Fields	Purpose
disaster_events	event_id (PK), hazard_type, state, district, start_date, status	Anchors every other table to a specific disaster response
shelters	shelter_id (PK), name, lat, lon, state, district, capacity, current_occupancy, hazard-suitability flags, status	Suitable shelters discovered in Stage 1, with live occupancy
population_groups	group_id (PK), event_id (FK), location, headcount, vulnerability_flag, status, timestamp	Displaced groups awaiting or holding an allocation
allocations	allocation_id (PK), group_id (FK), shelter_id (FK), people_count, distance_km, allocated_at	Immutable audit trail of every allocation made

6. Known Limitations

- Rural and village-level road mapping in OpenStreetMap is inconsistent across India, which is why haversine fallback distance exists alongside routed distance.
- Building structural material tagging (relevant to cyclone and earthquake suitability) is sparse in OSM for rural regions, making those two hazard modules less precise than the flood module for this review cycle.
- Area-based shelter capacity is a planning estimate derived from building footprint and an accepted space-per-person standard, not a guaranteed real-world number — actual usable capacity depends on internal room layout that public data cannot capture.
- The system currently runs on SQLite, which is appropriate for a single-instance prototype but not for concurrent multi-user, multi-district production use.
- No offline-first sync or authentication layer exists yet — both are necessary before any real field deployment.

7. Phased Roadmap

Phase	Scope
Phase 1 (this semester)	Rigorous prototype using real public datasets (Bhuvan, Census, IMD, OSM), SQLite persistence, Streamlit dashboard, backtested against a real Assam flood event.
Phase 2	Migration to PostgreSQL with PostGIS, offline-sync capability, role-based access control, live API integration in place of static data pulls.
Phase 3	Pilot engagement with a district-level State Disaster Management Authority (SDMA), incorporating real coordinator feedback and moving toward NDMA-compatible data standards.

Positioning Statement

The system is designed as a nationally-scalable, multi-hazard shelter allocation platform, with Assam flood response serving as the pilot and demonstration case due to its high disaster frequency and public data availability. No region-specific or hazard-specific logic is hardcoded into the core pipeline — only the data sources and suitability rules change per deployment.